DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 7: TRANSACTIONS & CONCURRENCY CONTROL

Question Bank with Comprehensive Answers — ACID, States, Serializability, Precedence Graphs, 2PL, Timestamp Protocol, Deadlock

SYLLABUS OVERVIEW — Unit 7

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages & Model	7 hrs	12
4	Constraints & Normalization	6 hrs	12
5	Query Processing & Optimization	4 hrs	8
6	File Structure & Hashing	4 hrs	8
7	Transactions & Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 7 Topics (6 Hours | 12 Marks)

- 7.1 Transaction Concepts & ACID Properties
- 7.2 Transaction States
- 7.3 Concurrent Execution — Problems
- 7.4 Schedules and Serializability (Conflict & View)
- 7.5 Recoverability
- 7.6 Lock-Based Protocols — Shared/Exclusive Locks, 2PL
- 7.7 Deadlock — Prevention, Detection, Recovery
- 7.8 Timestamp-Based Concurrency Control

CHAPTER 7 — Questions & Comprehensive Answers

Q: What is a transaction? Explain the ACID properties of a transaction with examples.
Asked: ALL years 2070–2082 — asked in every single exam | Marks: [4]

Answer

Transaction

A transaction is a logical unit of work that consists of a sequence of database operations (reads and writes) that must be executed atomically and in isolation. A transaction takes the database from one consistent state to another. It is initiated by a user or application program and ends with a COMMIT (make changes permanent) or ROLLBACK (undo all changes).

```
Example transaction: Transfer Rs.500 from Account A to Account B
  T1: read(A)
  T1: A := A - 500
  T1: write(A)
  T1: read(B)
  T1: B := B + 500
  T1: write(B)
  T1: COMMIT

All six steps must succeed together — or none should persist.
```

ACID Properties

1. Atomicity

A transaction is treated as an indivisible unit. Either ALL of its operations are executed and committed, or NONE of them take effect. If a transaction fails partway through, all its changes are rolled back as if the transaction never happened.

```
Example: Transfer Rs.500 from A to B
  If system crashes after write(A) but before write(B):
  → A has been debited but B not credited — inconsistency!
  Atomicity: rollback write(A) → database is restored to original state
  Property enforced by: Transaction Manager (undo logs)
```

2. Consistency

A transaction must take the database from one consistent (valid) state to another consistent state. All integrity constraints, business rules, and data relationships must be satisfied before and after the transaction — even if intermediate states during execution may be temporarily inconsistent.

```
Example: Transfer Rs.500 from A to B
  Before: A=1000, B=500; Total = 1500
  After:  A=500,  B=1000; Total = 1500 ✓
  Consistency: sum of all account balances is preserved
  Property enforced by: Application logic + integrity constraints
```

3. Isolation

Concurrent transactions must execute as if they were running serially (one at a time). Each transaction is isolated from the effects of other concurrent transactions. Intermediate (uncommitted) states of a transaction must NOT be visible to other transactions.

```
Example: T1 transfers Rs.500 from A to B; T2 reads A+B simultaneously
  Without isolation: T2 might read A after debit but before credit → sees
A=500, B=500, total=1000 (wrong!)
  With isolation: T2 sees either {A=1000,B=500} or {A=500,B=1000} — never an
intermediate state
  Property enforced by: Concurrency control (locks, timestamps)
```

4. Durability

Once a transaction commits, its changes are permanent and persist even in the event of system failures (crashes, power outages). The committed data is recorded in non-volatile storage before the commit is acknowledged.

```
Example: After T1 commits the transfer:
  Even if the system crashes immediately after COMMIT:
  → The changes to A and B must survive system restart
  Property enforced by: Recovery manager (redo logs, write-ahead logging)
```

Property	Description	Enforced By
Atomicity	All or nothing — partial execution not allowed	Transaction Manager (undo logs, rollback)
Consistency	DB moves from one valid state to another valid state	Application + integrity constraints
Isolation	Concurrent transactions do not interfere with each other	Concurrency Control (locks, timestamps)
Durability	Committed changes persist despite system failures	Recovery Manager (redo logs, WAL)

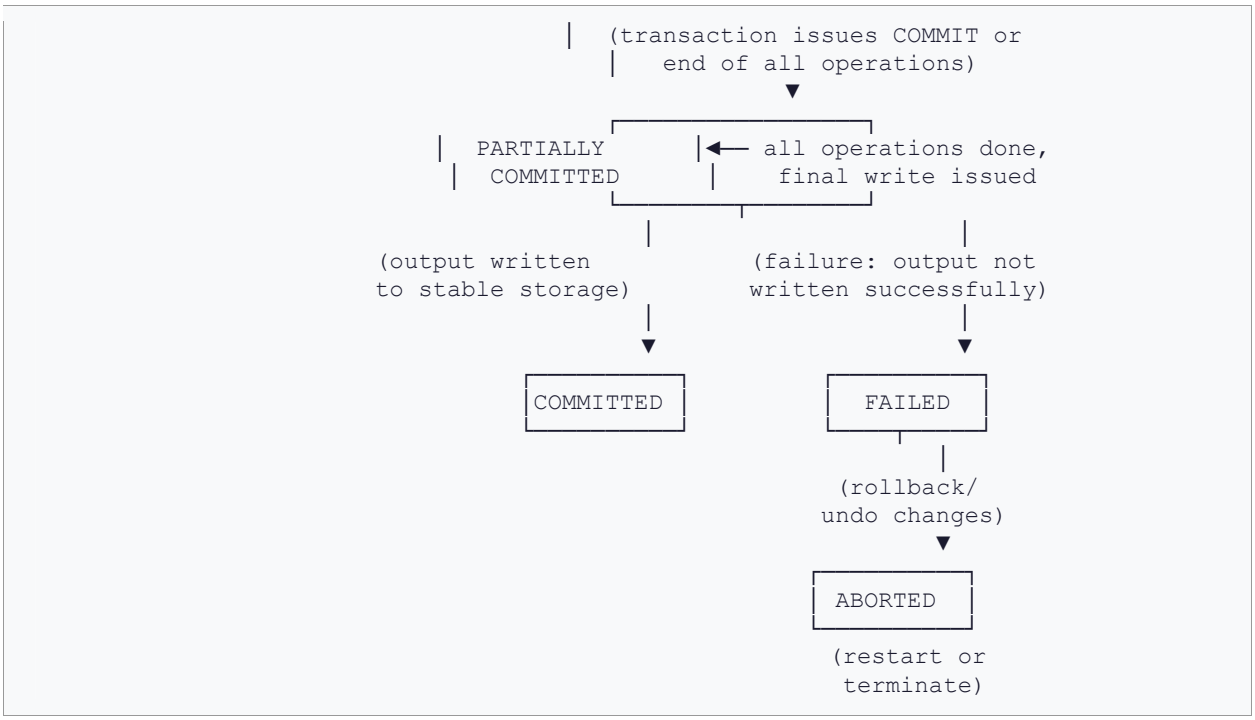
Q: Explain the transaction states with a state-transition diagram.
Asked: 2081 Chaitra; 2080 Ashwin; 2079 Jestha; 2078 Baishakh; 2076; 2073 Magh; 2071 Magh; 2070 Magh | Marks: [4]

Answer

Transaction States

A transaction passes through a series of states during its lifetime. The state-transition diagram shows how a transaction progresses from initiation to final completion.

```
                                  ┌──────────────┐
BEGIN ──────────►|   ACTIVE    |──────── read/write operations
                                  └──────┬───────┘
```



Description of Each State

1. Active

The transaction is currently being executed. It is performing read and write operations on the database. This is the initial state after BEGIN TRANSACTION.

2. Partially Committed

The transaction has executed its final operation (all read/write operations are complete). The changes are still in the volatile buffer (not yet written to stable disk storage). The DBMS checks if the output can be safely written to stable storage.

3. Committed

The transaction has successfully completed. All its changes have been permanently written to stable storage (disk). The changes are now visible to other transactions. The transaction cannot be rolled back from this state.

4. Failed

The transaction cannot proceed with normal execution. This happens due to errors (system crash, constraint violation, deadlock, user abort). The transaction must be rolled back.

5. Aborted

The transaction has been rolled back — all its changes have been undone and the database is restored to the state before the transaction began. The DBMS can either restart the transaction (if it failed due to a system error) or terminate it (if it failed due to an internal/logical error).

State	Description	Next Possible State

Active	Executing read/write operations	Partially Committed or Failed
Partially Committed	All operations done; writing to stable storage	Committed or Failed
Committed	Successfully completed; changes permanent	Terminal state (done)
Failed	Cannot proceed; must undo	Aborted
Aborted	Rolled back; changes undone	Restarted or Terminated

Q: What are the problems of concurrent execution? Explain dirty read, lost update, non-repeatable read, and phantom read with examples.
Asked: 2081 Chaitra; 2079 Chaitra; 2079 Jestha; 2078 Poush; 2078 Baishakh; 2076; 2072 Magh; 2070–2082 | Marks: [4]

Answer

Concurrent Execution

When multiple transactions execute concurrently (interleaved), their operations can interfere with each other in ways that produce incorrect or inconsistent results — even when each individual transaction is correct in isolation. There are four classic concurrency problems:

Problem 1: Dirty Read (Uncommitted Dependency)

T2 reads data that T1 has written but not yet committed. If T1 is later rolled back, T2 has read data that never officially existed — a 'dirty' (invalid) read.

```
T1                          T2
──────────────────────────────────────────
read(A)  {A=500}
A := A + 200
write(A) {A=700}
                            read(A)  {A=700}  ← reads UNCOMMITTED value
                            B := A * 2        (B = 1400 — wrong!)
ROLLBACK {A restored to 500}
                            write(B) {B=1400} ← based on invalid data
──────────────────────────────────────────
T2 has read dirty (uncommitted) data from T1. If T1 rollbacks, T2's result is
wrong.
```

Problem 2: Lost Update

Two transactions read the same data item and then both update it. One update overwrites the other, so one of the updates is lost.

```
T1                          T2
──────────────────────────────────────────
read(A)  {A=500}
                            read(A)  {A=500}
A := A + 200
write(A) {A=700}
                            A := A + 100
                            write(A) {A=600}  ← OVERWRITES T1's update!
──────────────────────────────────────────
```

```
T1's update (A=700) is lost. Final value A=600, but should be A=800.
T1's work is completely lost — update anomaly.
```

Problem 3: Non-Repeatable Read (Inconsistent Read / Fuzzy Read)

T1 reads a data item, then T2 modifies and commits it. If T1 reads the same data item again, it gets a different value — the same read gives inconsistent results within the same transaction.

```
T1                          T2
──────────────────────────────────────────
read(A)  {A=500}
                            read(A)  {A=500}
                            A := A + 200
                            write(A) {A=700}
                            COMMIT
read(A)  {A=700}  ← different from first read!
──────────────────────────────────────────
T1 reads A twice and gets different values (500 then 700) within the same
transaction.
The read is not repeatable — violates isolation.
```

Problem 4: Phantom Read

T1 executes a query that returns a set of rows satisfying a condition. T2 then inserts or deletes rows that satisfy the same condition. If T1 re-executes the same query, it gets a different set of rows — phantom rows appear or disappear.

```
T1                              T2
──────────────────────────────────────────────────
SELECT * FROM Emp
WHERE salary > 50000  → {E1,E2,E3}
                                T2: INSERT new employee E4 with salary=60000
                                T2: COMMIT
SELECT * FROM Emp
WHERE salary > 50000  → {E1,E2,E3,E4}  ← E4 is a PHANTOM!
──────────────────────────────────────────────────
T1's second query returns an extra row E4 that wasn't there before.
Not caused by modifying existing data, but by new rows being inserted.
```

Problem	Cause	Example Effect
Dirty Read	Reading uncommitted data of another transaction	T2 reads T1's data; T1 rollbacks → T2 used invalid data
Lost Update	Two transactions update same item; one overwrites the other	A=500; T1 and T2 both add 200 and 100; final A=600 instead of 800
Non-Repeatable Read	Another transaction modifies and commits data between two reads	T1 reads A=500, then A=700 in same transaction
Phantom Read	Another transaction inserts/deletes rows matching a query condition	T1's COUNT query returns different results on two executions

Q: What is serializability? Explain conflict serializability and view serializability. What is a precedence (serialization) graph?

Asked: ALL years 2070–2082 — core topic, high marks | Marks: [4–8]

Answer

Schedule and Serializability

A schedule S is a sequence of read and write operations from a set of concurrent transactions. A serial schedule is one where transactions execute one at a time with no interleaving. A schedule is serializable if it is equivalent (produces the same final database state) to some serial schedule.

```
Serial schedule example (T1 then T2):
  r1(A); w1(A); r1(B); w1(B); r2(A); w2(A); r2(B); w2(B)

Concurrent (interleaved) schedule:
  r1(A); r2(A); w1(A); w2(A); r1(B); w2(B); w1(B)
  → Is this equivalent to a serial schedule? Need to test serializability.
```

1. Conflict Serializability

Conflicting Operations:

Two operations from different transactions on the same data item CONFLICT if at least one of them is a write:

```
Given operations Oi from Ti and Oj from Tj (i ≠ j) on same data item Q:
  • read_i(Q) and read_j(Q)  → NO conflict (both just read)
  • read_i(Q) and write_j(Q) → CONFLICT
  • write_i(Q) and read_j(Q) → CONFLICT
  • write_i(Q) and write_j(Q)→ CONFLICT
```

Conflict Equivalent Schedules:

Two schedules S1 and S2 are conflict equivalent if S2 can be obtained from S1 by swapping the order of non-conflicting operations. A schedule is conflict serializable if it is conflict equivalent to some serial schedule.

2. Precedence Graph (Serialization Graph)

The precedence graph is the standard tool for testing conflict serializability:

```
Construction:
  • One node per transaction Ti
  • Draw edge Ti → Tj if there exists a conflicting pair (Oi, Oj) where Oi
comes before Oj in S
    Specifically, draw Ti → Tj if:
      - Ti executes write(Q) before Tj executes read(Q), OR
      - Ti executes read(Q) before Tj executes write(Q), OR
      - Ti executes write(Q) before Tj executes write(Q)

Test:
  Schedule S is conflict serializable □ the precedence graph has NO cycle
  If the graph is acyclic, a topological sort gives the equivalent serial
order
```

3. View Serializability

View serializability is a broader definition. Two schedules S1 and S2 are view equivalent if:

- For each data item Q, if Ti reads the initial value of Q in S1, Ti also reads the initial value of Q in S2
- For each read(Q) by Ti in S1: if Ti reads the value written by Tj, then Ti also reads Tj's value in S2
- For each data item Q, the transaction that performs the final write(Q) in S1 also performs the final write(Q) in S2

Every conflict serializable schedule is also view serializable (conflict □ view).
But NOT vice versa — a view serializable schedule may not be conflict serializable.
View serializability is harder to test (NP-complete in general).
In practice, conflict serializability (precedence graph) is used.

Q: Draw the precedence graph for schedule S = r1(A); r2(A); w1(A); r1(B); w2(A); w1(B); w2(B). Determine if it is conflict serializable. If yes, find the equivalent serial schedule.
Asked: 2082 Kartik (Back) — most recent exam | Marks: [8]

Answer — Precedence Graph Analysis

Schedule S:

```
Step  Operation   Item
  1   r1(A)        A     ← T1 reads A
  2   r2(A)        A     ← T2 reads A
  3   w1(A)        A     ← T1 writes A
  4   r1(B)        B     ← T1 reads B
  5   w2(A)        A     ← T2 writes A
  6   w1(B)        B     ← T1 writes B
  7   w2(B)        B     ← T2 writes B
```

Step 1: Identify all conflicting pairs

Pair	Operations	Data Item	Conflict?	Edge
(1,5)	r1(A) before w2(A)	A	YES — read then write	T1 → T2
(2,3)	r2(A) before w1(A)	A	YES — read then write	T2 → T1
(3,5)	w1(A) before w2(A)	A	YES — write then write	T1 → T2
(4,7)	r1(B) before w2(B)	B	YES — read then write	T1 → T2
(6,7)	w1(B) before w2(B)	B	YES — write then write	T1 → T2

(1,3)	r1(A) before w1(A)	A	SAME tx — ignore	—
(2,5)	r2(A) before w2(A)	A	SAME tx — ignore	—

Step 2: Build the Precedence Graph

```
                    Edges found:
T1 → T2  (from pairs (1,5), (3,5), (4,7), (6,7))
          T2 → T1  (from pair (2,3))

                Precedence Graph:

                  ┌──────────────────┐
                  │                  │
                  ▼                  │
                 ┌──┐   T1→T2       ┌──┐
                 │T1│ ──────────→   │T2│
                 └──┘                └──┘
                  ▲                  │
                  │    T2→T1         │
                  └──────────────────┘

        There is a CYCLE: T1 → T2 → T1
```

Step 3: Conclusion

The precedence graph contains a cycle (T1 → T2 → T1). Therefore, schedule S is NOT conflict serializable.

The cycle means: T1 must come before T2 (due to edges T1→T2) AND T2 must come before T1 (due to T2→T1).
This is a contradiction — no valid serial ordering exists.
The schedule allows a dirty read anomaly (T2 reads A at step 2 before T1's write at step 3, but T2 writes A at step 5 after T1 writes at step 3).

Q: Check if schedule S: r1(X); r2(X); r1(Y); r3(Y); r2(Y); w1(X); w3(Y); r2(Z); w2(Z) is conflict serializable using a precedence graph.
Asked: 2081 Ashwin (Back Exam) | Marks: [8]

Answer — Precedence Graph Analysis

Schedule S (labelled):

```
Step  Operation  Item
  1   r1(X)       X
  2   r2(X)       X
  3   r1(Y)       Y
  4   r3(Y)       Y
  5   r2(Y)       Y
  6   w1(X)       X
```

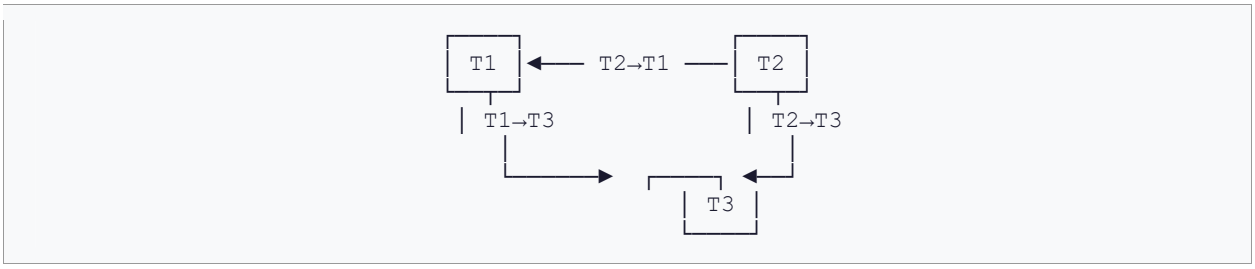
```
  7   w3(Y)        Y
  8   r2(Z)        Z
  9   w2(Z)        Z
```

Step 1: Find all conflicting pairs (different transactions, same item, ≥1 write)

Pair	Operations	Item	Edge
(1,6)	r1(X) before w1(X)	X	same T1 — ignore
(2,6)	r2(X) before w1(X)	X	T2 → T1
(3,7)	r1(Y) before w3(Y)	Y	T1 → T3
(4,7)	r3(Y) before w3(Y)	Y	same T3 — ignore
(5,7)	r2(Y) before w3(Y)	Y	T2 → T3
(4,5)	r3(Y) before r2(Y)	Y	both reads — no conflict
(8,9)	r2(Z) before w2(Z)	Z	same T2 — ignore

Edges to add: T2→T1, T1→T3, T2→T3

Step 2: Build the Precedence Graph



Step 3: Check for cycles

Edges: T2→T1, T1→T3, T2→T3. There is no path from T1 to T2, no path from T3 to T1 or T2. The graph is ACYCLIC.

Step 4: Conclusion + Topological Sort

The precedence graph is acyclic — therefore schedule S IS conflict serializable.

```
Topological sort (serial order):
  T2 has no incoming edges → T2 goes first
  T1 has incoming edge only from T2 → T1 goes second
  T3 has incoming edges from T1 and T2 → T3 goes last

Equivalent serial schedule: T2 → T1 → T3

Verify: Serial schedule T2,T1,T3 should give same result as S.
```

Q: Use a precedence graph to test conflict serializability of: r1(X); r2(Z); r1(Z); r3(X); r3(Y); w1(X); w3(Y); r2(Y); w2(Z); w2(Y)
Asked: 2078 Chaitra (Regular Exam) | Marks: [6]

Answer — Precedence Graph Analysis

Schedule S:

```
Step  Op     Item
  1   r1(X)   X
  2   r2(Z)   Z
  3   r1(Z)   Z
  4   r3(X)   X
  5   r3(Y)   Y
  6   w1(X)   X
  7   w3(Y)   Y
  8   r2(Y)   Y
  9   w2(Z)   Z
 10   w2(Y)   Y
```

Conflicting pairs and edges:

Conflict	Op_i before Op_j	Edge
X: r1(X) vs w3(Y)	No — different items	—
X: r3(X) before w1(X)	r3(X)[4] before w1(X)[6]	T3 → T1
X: r1(X) before... no write by T2/T3 after step 1 on X	r1(X)[1], w3? no, w2? no	—
Z: r2(Z) before w2(Z)	same T2 — ignore	—
Z: r1(Z) before w2(Z)	r1(Z)[3] before w2(Z)[9]	T1 → T2
Z: r2(Z) before... w1? no w on Z by T1	—	—
Y: r3(Y) before w2(Y)	r3(Y)[5] before w2(Y)[10]	T3 → T2
Y: r2(Y) before w3(Y)	r2(Y)[8] after w3(Y)[7] — wrong order, no conflict here	—
Y: w3(Y) before r2(Y)	w3(Y)[7] before r2(Y)[8]	T3 → T2
Y: w3(Y) before w2(Y)	w3(Y)[7] before w2(Y)[10]	T3 → T2 (already have)
X: w1(X) — any reads/writes on X after?	w1(X)[6], r3(X)[4] is before — already noted	—

Edges summary: T3→T1, T1→T2, T3→T2

```
┌────┐
│ T3 │──T3→T1──────▶│ T1 │
└────┘              └────┘
 │ T3→T2              │ T1→T2
 │                    │
 └──────▶ ┌────┐ ◀────┘
          │ T2 │
          └────┘
```

No cycle exists (T3 has no incoming edges; T2 has no outgoing edges). The graph is ACYCLIC.

```
Equivalent serial order (topological sort): T3 → T1 → T2
Schedule is CONFLICT SERIALIZABLE ✓
```

Q: Construct a precedence graph for: r1(A); r2(A); r3(A); w1(A); r2(B); w2(B); w3(B); r1(B). Is it conflict serializable?

Asked: 2079 Ashwin (Back Exam) | Marks: [4]

Answer — Precedence Graph

Schedule:

```
Step  Op     Item
  1   r1(A)   A
  2   r2(A)   A
  3   r3(A)   A
  4   w1(A)   A
  5   r2(B)   B
  6   w2(B)   B
  7   w3(B)   B
  8   r1(B)   B
```

Conflicting pairs:

```
On A:
  r2(A)[2] before w1(A)[4]  → T2 → T1
  r3(A)[3] before w1(A)[4]  → T3 → T1

On B:
  r2(B)[5] before w3(B)[7]  → T2 → T3
  w2(B)[6] before w3(B)[7]  → T2 → T3  (already have T2→T3)
  w2(B)[6] before r1(B)[8]  → T2 → T1
  w3(B)[7] before r1(B)[8]  → T3 → T1
```

```
Edges: T2→T1, T3→T1, T2→T3

T2 ────T2→T1──────────────▶ T1
 │                           ▲
 │  T2→T3                    │
```

```
└──────────────▶ T3 ──T3→T1──┘
```

No cycle: T2 has no incoming edges; T1 has no outgoing edges. The graph is ACYCLIC.

```
Topological sort: T2 → T3 → T1
Schedule IS conflict serializable. Equivalent serial order: T2, T3, T1.
```

Q: Explain lock-based concurrency control protocols. Describe shared lock, exclusive lock, and the lock compatibility matrix.
Asked: 2079 Ashwin; 2075 Baisakh; 2073 Magh; 2071 Magh; 2070–2082 | Marks: [4]

Answer

Lock-Based Concurrency Control

Locking is the most widely used mechanism for concurrency control. Before accessing a data item, a transaction must first acquire a lock on it. The lock type determines what access is permitted and whether other transactions can access the same item simultaneously.

Types of Locks

1. Shared Lock (S-lock / Read Lock)

A shared lock allows the transaction to READ the data item. Multiple transactions can hold shared locks on the same data item simultaneously — reads do not conflict with each other.

```
lock-S(Q): acquire shared lock on Q
unlock(Q): release the lock

If Ti holds S-lock on Q:
  Other transactions can also acquire S-lock on Q ✓
  Other transactions CANNOT acquire X-lock on Q ✗
```

2. Exclusive Lock (X-lock / Write Lock)

An exclusive lock allows the transaction to both READ and WRITE the data item. Only one transaction can hold an exclusive lock on a data item at any time — no other lock (S or X) can be granted while an X-lock is held.

```
lock-X(Q): acquire exclusive lock on Q
unlock(Q): release the lock

If Ti holds X-lock on Q:
  Other transactions CANNOT acquire S-lock on Q ✗
  Other transactions CANNOT acquire X-lock on Q ✗
```

Lock Compatibility Matrix

Lock Requested →	S (Shared)	X (Exclusive)

S already held	Compatible ✓ (grant)	Incompatible ✗ (wait)
X already held	Incompatible ✗ (wait)	Incompatible ✗ (wait)
No lock held	Compatible ✓ (grant)	Compatible ✓ (grant)

Lock Protocol Rules

- A transaction must acquire lock-S(Q) or lock-X(Q) before reading Q
- A transaction must acquire lock-X(Q) before writing Q
- A transaction may upgrade a shared lock to exclusive (lock upgrade) if no other transaction holds S-lock on Q
- A transaction releases locks via unlock(Q) — the timing of unlocks is governed by the locking protocol

Locking with Transactions

```
Example: T1 reads A and writes B; T2 reads A and writes A

T1                              T2
lock-S(A)
read(A)
                                lock-S(A)   ← granted (S compatible with S)
                                read(A)
lock-X(B)
write(B)
unlock(A)
unlock(B)
                                lock-X(A)   ← now granted (T1 released S)
                                write(A)
                                unlock(A)
```

Q: Explain the two-phase locking (2PL) protocol. Differentiate basic 2PL, strict 2PL, and rigorous 2PL. What is the problem with basic 2PL?
Asked: 2081 Chaitra; 2080; 2079 Jestha; 2078 Poush; 2078 Baishakh; 2077; 2075; 2074; 2073; 2072; 2071; 2070 — all years | Marks: [4]

Answer

Two-Phase Locking (2PL) Protocol

The two-phase locking protocol is the most widely used concurrency control protocol. It ensures conflict serializability by requiring that all lock acquisitions occur before any lock release. It divides a transaction's execution into exactly two phases:

```
                                    Number
                                      of
                                    Locks
                                      ▲
        |          ┌──────── Growing Phase ────────┐   ┌── Shrinking Phase ──┐
```

```
                  |         /                                  \/
                  |        /                                   /\
        |       /   Acquire locks only              /  \   Release locks only
        |      /     (no releases allowed)         /    \  (no acquisitions)
                |     /                                   /      \
          └──────────────────────────────────────────────────────────► Time
          BEGIN           Lock Point (max locks)              COMMIT/ABORT
```

Two Phases

Phase 1: Growing Phase

The transaction may acquire new locks (S or X) but may NOT release any lock. Locks are acquired as data items are needed. This phase ends at the lock point — the moment the transaction acquires its last lock.

Phase 2: Shrinking Phase

The transaction may release locks but may NOT acquire any new locks. Once a transaction releases its first lock, it enters the shrinking phase and can never acquire another lock.

Variants of 2PL

1. Basic (Standard) 2PL

Locks are released as soon as they are no longer needed (once the item has been read/written). The transaction may release locks before COMMIT.

- Guarantees: conflict serializability
- Problem: Cascading rollbacks (see below)

2. Strict 2PL

All EXCLUSIVE locks are held until the transaction commits or aborts. Shared locks may be released during the transaction.

- Guarantees: conflict serializability + no cascading rollbacks for dirty reads (strict schedule)
- Most common variant used in real DBMS
- Prevents dirty reads because X-locks are held until commit

3. Rigorous 2PL

ALL locks (both shared and exclusive) are held until the transaction commits or aborts. The most conservative variant.

- Guarantees: conflict serializability + no cascading rollbacks at all
- Easiest to implement — just release all locks at COMMIT
- More blocking than strict 2PL — lower concurrency

Protocol	When Locks Released	Cascading Rollback	Concurrency
Basic 2PL	S and X locks released during execution	Possible ✗	Highest
Strict 2PL	X locks held until commit/abort	Not possible ✓	Medium
Rigorous 2PL	All locks held until commit/abort	Not possible ✓	Lowest

Problem: Cascading Rollbacks with Basic 2PL

In basic 2PL, a transaction may release an exclusive lock before committing. If another transaction reads this (now unlocked but still uncommitted) data and then the first transaction aborts, the second transaction has read invalid data and must also be rolled back — and so on in a cascade.

```
T1                      T2
lock-X(A)
write(A)
unlock(A)  ← releases before commit
                        lock-S(A)
                        read(A)   ← reads T1's uncommitted value
                        unlock(A)
ABORT  ← T1 rolls back A
                        → T2 must also ABORT (cascading rollback!)
```

Strict 2PL prevents this: T1 holds X-lock until COMMIT, so T2 cannot read A until T1 has committed.

Q: What is a deadlock? Explain deadlock prevention (wait-die and wound-wait), deadlock detection (waits-for graph), and recovery from deadlock.

Asked: 2080; 2078 Chaitra; 2078 Baishakh; 2076; 2073 Magh; 2071 Magh; 2070–2082 | Marks: [4]

Answer

Deadlock

A deadlock occurs when two or more transactions are in a circular wait — each is waiting for a lock held by another in the cycle. None of them can proceed, and the system is stuck indefinitely.

```
Classic deadlock example:

T1                          T2
lock-X(A)  ← holds A
                            lock-X(B)  ← holds B
lock-X(B)  ← WAITS for B
                            lock-X(A)  ← WAITS for A

T1 waits for T2 to release B
T2 waits for T1 to release A
→ DEADLOCK — neither can proceed
```

1. Deadlock Prevention

Prevention ensures deadlocks never occur by imposing restrictions on how transactions request locks. Two timestamp-based schemes are widely used:

Timestamps

Each transaction Ti is assigned a unique timestamp TS(Ti) when it begins. Older transactions have smaller timestamps. If TS(T1) < TS(T2), T1 is older than T2.

Wait-Die Scheme (Non-preemptive)

When Ti requests a lock held by Tj:

- If Ti is OLDER than Tj (TS(Ti) < TS(Tj)): Ti WAITS (older transaction waits for younger)
- If Ti is YOUNGER than Tj (TS(Ti) > TS(Tj)): Ti DIES (aborted and restarted with same original timestamp)

```
Wait-Die: Only OLDER transactions wait; younger ones are killed (die).

Example: TS(T1)=5, TS(T2)=10, TS(T3)=15
  T1 requests lock held by T2: T1 older → T1 WAITS
  T3 requests lock held by T2: T3 younger → T3 DIES (aborted)
  T2 requests lock held by T1: T2 younger → T2 DIES
```

Wound-Wait Scheme (Preemptive)

When Ti requests a lock held by Tj:

- If Ti is OLDER than Tj (TS(Ti) < TS(Tj)): Ti WOUNDS Tj — Tj is aborted (preempted) and Ti gets the lock
- If Ti is YOUNGER than Tj (TS(Ti) > TS(Tj)): Ti WAITS for Tj

```
Wound-Wait: Older transactions preempt (wound) younger ones; younger ones
wait.

Example: TS(T1)=5, TS(T2)=10
  T1 requests lock held by T2: T1 older → T1 WOUNDS T2 (T2 aborted)
  T2 requests lock held by T1: T2 younger → T2 WAITS
```

Scheme	Older requests lock held by Younger	Younger requests lock held by Older	Direction
Wait-Die	WAIT	DIE (abort)	Older waits; younger dies
Wound-Wait	WOUND (preempt younger)	WAIT	Older wounds; younger waits

2. Deadlock Detection — Waits-For Graph

Rather than preventing deadlocks, the DBMS periodically checks for their existence using a waits-for graph:

```
Waits-For Graph:
  • One node per active transaction
  • Draw edge Ti → Tj if Ti is waiting for a lock held by Tj

Test: A deadlock exists □ the waits-for graph has a CYCLE

Example:
  T1 waits for T2 (T1 → T2)
  T2 waits for T3 (T2 → T3)
  T3 waits for T1 (T3 → T1)
  Cycle: T1 → T2 → T3 → T1  → DEADLOCK
```

3. Deadlock Recovery

When a deadlock is detected, the DBMS must break the cycle by aborting one or more transactions (the victim):

Victim Selection Criteria:

- Select the transaction that has done the least work (fewest updates)
- Select the youngest transaction (fewest resources committed)
- Select the transaction that can be rolled back with the least cost

Rollback:

- Total rollback: abort the entire victim transaction and restart it
- Partial rollback: roll back only as far as needed to break the deadlock (requires savepoints)

Starvation prevention:

The same transaction should not always be chosen as victim. Include the number of times a transaction has been rolled back as a cost factor — if a transaction has been rolled back too many times, give it higher priority.

Q: Explain the timestamp-based concurrency control protocol. How does it handle read and write operations?

Asked: 2079 Jestha; 2074; 2072 Magh; 2070 Magh | Marks: [4]

Answer

Timestamp-Based Concurrency Control

The timestamp ordering protocol is a non-lock-based concurrency control method. Each transaction Ti is assigned a unique timestamp TS(Ti) at its start. The protocol ensures that conflicting operations are executed in timestamp order, guaranteeing serializability equivalent to the serial schedule in timestamp order.

Timestamp Values per Data Item

For each data item Q, the DBMS maintains two timestamp values:

```
W-timestamp(Q) = largest timestamp of any transaction that successfully wrote
Q
R-timestamp(Q) = largest timestamp of any transaction that successfully read Q
```

Protocol Rules

When Ti issues read(Q):

```
Case 1: TS(Ti) < W-timestamp(Q)
  → Ti needs to read a value that was already OVERWRITTEN by a newer
transaction
  → Ti is too old — it has missed its read window
  → ABORT Ti and restart with a newer timestamp

Case 2: TS(Ti) ≥ W-timestamp(Q)
  → Safe to read: Ti reads Q
  → Update R-timestamp(Q) = max(R-timestamp(Q), TS(Ti))
```

When Ti issues write(Q):

```
Case 1: TS(Ti) < R-timestamp(Q)
  → A newer transaction has already READ Q
```

```
  → Ti's write would affect a read that has already happened
  → ABORT Ti and restart with a newer timestamp

Case 2: TS(Ti) < W-timestamp(Q)
  → A newer transaction has already WRITTEN Q
  → Ti's write is 'too old' — its value is already obsolete
  → Thomas' Write Rule: Ti's write can be IGNORED (not a conflict, just skip)
     (otherwise: ABORT Ti)

Case 3: TS(Ti) ≥ R-timestamp(Q) AND TS(Ti) ≥ W-timestamp(Q)
  → Safe to write: Ti writes Q
  → Update W-timestamp(Q) = TS(Ti)
```

Properties of Timestamp Protocol

- No locks needed — avoids deadlocks entirely
- Guarantees serializability (equivalent to serial order by timestamp)
- Starvation possible: a transaction may be repeatedly aborted if it keeps conflicting with newer transactions
- Timestamps are assigned at start — no growing/shrinking phases

Aspect	Timestamp Protocol	2PL
Mechanism	Timestamps — no locks	Locks (S, X)
Deadlock	Impossible — no waiting	Possible — waiting for locks
Starvation	Possible — transaction may keep aborting	Possible — transactions may wait indefinitely
Serializability	By timestamp order	By lock order
Overhead	Maintain R/W timestamps per data item	Maintain lock table

---

Q: What is recoverability? Explain recoverable schedules, cascadeless schedules, and strict schedules.

Asked: 2075 Baisakh (Back Exam) | Marks: [4]

Answer

Recoverability of Schedules

A schedule must not only be serializable but also recoverable — if a transaction fails, the DBMS must be able to undo its effects without corrupting data that other transactions have already committed.

1. Recoverable Schedule

A schedule S is recoverable if: for every pair of transactions Ti and Tj where Tj reads a data item written by Ti, the commit of Ti appears before the commit of Tj in S.

```
Rule: If Tj reads from Ti, then Ti must COMMIT before Tj commits.

Non-recoverable (bad) example:
  T1: write(A)
  T2: read(A)    ← T2 reads T1's value
  T2: COMMIT     ← T2 commits before T1!
```

```
  T1: ABORT     ← now T2 has committed data based on aborted T1
  → Cannot undo T2's commit → NOT RECOVERABLE ✗

Recoverable example:
  T1: write(A)
  T2: read(A)
  T1: COMMIT    ← T1 commits first ✓
  T2: COMMIT    ← T2 commits after T1 ✓
```

2. Cascadeless Schedule

A cascadeless (cascade-free) schedule prevents cascading rollbacks. A schedule is cascadeless if: a transaction Tj may only read data items written by Ti after Ti has already committed.

```
Rule: Tj can only read Ti's writes AFTER Ti commits.

This prevents: Tj reads T1's uncommitted data → T1 aborts → Tj must also abort
  (cascading rollback chain)

Cascadeless □ Recoverable  (every cascadeless schedule is recoverable)
Recoverable does NOT imply cascadeless
```

3. Strict Schedule

A strict schedule is even more restrictive: a transaction Tj may neither read nor write a data item written by Ti until Ti has committed or aborted.

```
Rule: No reading OR writing of another transaction's uncommitted writes.

Strict □ Cascadeless □ Recoverable

Strict schedules allow simple recovery: just restore the before-image of the
data item.
Strict 2PL produces strict schedules (X-locks held until commit).
```

Schedule Type	Rule	Cascade Rollback	Recovery
Recoverable	Tj commits after Ti if Tj read from Ti	Possible	Possible but complex
Cascadeless	Tj reads Ti's data only after Ti commits	Not possible	Easier
Strict	Tj reads/writes Ti's data only after Ti commits/aborts	Not possible	Simplest — restore before-image

SUMMARY TABLE — Chapter 7 Question Coverage

All Chapter 7 questions from TU-IOE BCT examination papers (2070–2082) and their coverage in this document.

No.	Question / Topic	Asked In	Marks
C1	Transaction definition, ACID properties with examples	ALL years 2070–2082	[4]
C2	Transaction states & state-transition diagram	2081 Chaitra, 2080, 2079 Jestha, 2078 Baishakh, 2076, 2073 Magh, 2071 Magh, 2070 Magh	[4]
C3	Concurrent execution problems: dirty read, lost update, non-repeatable read, phantom	2081 Chaitra, 2079 Chaitra, 2079 Jestha, 2078 Poush, 2078 Baishakh, 2076, 2075–2070	[4]
C4	Schedules, serializability — conflict and view serializability	All years 2070–2082	[4–8]
C5	Precedence graph — 2082 schedule r1(A);r2(A);w1(A);r1(B);w2(A);w1(B);w2(B)	2082 Kartik	[8]
C6	Precedence graph — 2081 schedule r1(X);r2(X);r1(Y);r3(Y);r2(Y);w1(X);w3(Y);r2(Z);w2(Z)	2081 Ashwin	[8]
C7	Precedence graph — 2078 schedule r1(X);r2(Z);r1(Z);r3(X);r3(Y);w1(X);w3(Y);r2(Y);w2(Z);w2(Y)	2078 Chaitra	[6]
C8	Precedence graph — 2079 Ashwin r1(A);r2(A);r3(A);w1(A);r2(B);w2(B);w3(B);r1(B)	2079 Ashwin	[4]
C9	Lock-based protocols: shared lock, exclusive lock, lock compatibility matrix	2079 Ashwin, 2075 Baisakh, 2073 Magh, 2071 Magh, 2070–2082	[4]
C10	Two-phase locking (2PL): basic, strict, rigorous — phases, problems	2081 Chaitra, 2080, 2079 Jestha, 2078 Poush, 2078 Baishakh, 2077, 2075, 2074, 2073, 2072, 2071, 2070	[4]
C11	Deadlock: definition, prevention (wait-die, wound-wait), detection (waits-for graph), recovery	2080, 2078 Chaitra, 2078 Baishakh, 2076, 2073 Magh, 2071 Magh, 2070–2082	[4]
C12	Timestamp-based concurrency control protocol	2079 Jestha, 2074, 2072 Magh, 2070 Magh	[4]
C13	Recoverability: recoverable, cascadeless, strict schedules	2075 Baisakh	[4]

Key notation: ri(X) = transaction Ti reads X; wi(X) = Ti writes X. Precedence edge Ti→Tj means Ti's operation must come before Tj's. A cycle in the precedence graph means NOT conflict serializable. 2PL growing phase: acquire only. Shrinking phase: release only. Strict 2PL holds X-locks until commit. Wait-die: older waits, younger dies. Wound-wait: older wounds, younger waits.

— End of Chapter 7 Answer Bank —